

Unit Testing

Prove one small
piece of code is
right —
automatically, in
milliseconds.

team time • unit testing
~4 min

✓ proof price parity ✓ cart validation ✓ config guard

3 files • 7 tests • **passed** • 41 ms

▸ `// definition`

What is a unit test?

A unit test checks that **one small piece of code** — usually a single function — does the right thing, **by itself**, in a fraction of a second.

input → your function → output ...and the test checks that output is what you expected.

No database, no browser, no network, so it's tiny and instant. That's why it's the base of the **testing pyramid**: picture all your tests stacked up, lots of small fast ones like this at the bottom, only a few slow, full ones at the top.

▸ *// the pattern behind every test*

Three beats: Arrange · Act · Assert

In plain English: **Arrange** sets up the input, **Act** runs the one thing you're testing, **Assert** checks that what came back is what you expected.

Arrange

```
// the input  
const cents = 129900
```

Act

```
// run the one thing under test  
const label = formatPrice(cents)
```

Assert

```
// say what you expect – the robot checks it  
expect(label).toBe('$1,299.00')
```

Given this **input**, I expect this **output**. That's the whole job.

▸ ~/pricing/tests

This is our code — tested

● price-parity.test.js

```
// reads a file, returns its price number
// example: "...= 1000" captures 1000
const RE =
  /ADDON_PRICE_CENTS = (\d+)/
async function centsIn(path) {
  const source = await readFile(url(path))
  return Number(source.match(RE)[1])
}
```

● same file · the test

```
it('shown === billed', async () => {
  // arrange + act: read each file's price
  const shown = await centsIn('...store.js')
  const billed = await centsIn('...run.js')

  // assert: must be the exact same number
  expect(billed).toBe(shown)
})
```

`centsIn` opens a file and pulls out the number after `ADDON_PRICE_CENTS =` using that regex above it. Real file paths, real regex, real `await`, nothing mocked.

Same AAA pattern, just blended: reading the file **is** both the input and the thing being tested, so Arrange and Act happen in the same two lines. Assert still stands alone.

▸ `// a real bug that never happened`

A test that protects revenue

One price lived in **two files** — the label the shopper sees, and the amount we bill. A comment said *“keep in sync.”*

A comment is not a guard.

✗ drift → CI fails

✓ match → ships

Caught automatically by CI (our automated test runner) the moment the code is pushed, not by a customer noticing they were **overcharged**. That's the payoff: bugs found early & cheap · refactor without fear · tests document intent · seconds per run.

```
// shown price vs billed price
const shown = await centsIn('...store.js')
const billed = await centsIn('...run.js')

expect(billed).toBe(shown)
```

`npm test` runs here first in our deploy workflow: a red suite blocks the deploy.

▸ `// what makes a test good`

Good tests are **FIRST**

F Fast — milliseconds, so you run them all the time.

I Independent — no test needs another; any order works.

R Repeatable — same result every run: no real clock, network, or randomness.

S Self-validating — it passes or fails on its own; no eyeballing.

T Timely — write it with the code, while the logic is fresh.

Test **what a function gives back**, not how it's written inside, so the test still passes after you rewrite it later (a refactor). Easiest place to start: functions like `formatPrice` from earlier, plain input-to-output logic with nothing random, no dates, no network.

▸ `// watch out for these`

Common mistakes

- ✗ **Testing how it works, not what it does.** → **check that an input gives the right output, so tests survive a refactor.**
- ✗ **Flaky tests** — a real clock, `Math.random()`, or the network. → **fake them; same result every run.**
- ✗ **Tests that lean on each other.** → **each one stands alone; reset shared state in `beforeEach`.**
- ✗ **A test that can't fail** — no real `expect`, or just `toBeDefined()`. → **check the actual value you expect.**
- ✗ **One test checking twenty different things.** → **one thing per test, with a name that says exactly what broke.**

Spot the pattern? Every mistake here is just a broken FIRST rule (slide before).

▸ `// when do I actually write one`

Good fit vs. someone else's topic

✓ Two numbers that must match

`price-parity.test.js` checks that the price the shopper sees and the price we bill are still the exact same number.

✓ Real examples run through the code

`fixtures.test.js` feeds sample shopping-cart data through the actual pricing and validation code, to prove it works.

✓ Catching bad data before it ships

Our spreadsheet import tooling has tests that catch a broken spreadsheet row before it ever reaches a real store.

✗ Not this talk

Testing a full page or user flow is Sara's **integration** talk. Confirming old features still work after a change is John's **regression** talk.

The gut check: input goes into your function, output comes out, no browser, database, or network in between. If that's true, it's a unit test.

▸ `$ npm test`

Try it Monday

- We run **Vitest** on the pricing app. Test locally: `cd` into the extension folder → `npm ci && npm test` — the same command CI runs before every deploy.
- Pick a function you own. Add a file in its `tests/` folder: `yourThing.test.js`
- Write one `it(...)`: arrange an input, act, expect the output. Async? Just add `async/await`, nothing else changes.

✓ CV: “writes unit tests (Vitest / Jest)”

One function. One check. A robot that runs it forever, for free.

▸ `// cheat sheet – screenshot this`

Vitest in one screen

• `tests/formatPrice.test.js` • a sibling `tests/` folder

```
import { describe, it, expect } from 'vitest'
import { formatPrice } from '../formatPrice'

describe('formatPrice', () => {
  it('formats cents as dollars', () => {
    // Arrange
    const cents = 129900
    // Act
    const label = formatPrice(cents)
    // Assert
    expect(label).toBe('$1,299.00')
  })
})
```

Matchers you'll actually use

```
.toBe(x) // ===
.toEqual(obj) // deep
.toContain(x)
.toThrow()
.toBeNull() • .toBeTruthy()
```

Commands

```
npm test // run once
npx vitest // watch
npx vitest --coverage
```

▸ // level up

When you're ready for more

Test doubles (mocks & stubs)

Fake the slow or unpredictable stuff — network, time, randomness — so tests stay Fast & Repeatable.

Coverage

A map of untested lines. A guide, not a goal — 100% of trivial code proves little.

TDD (Test-Driven Development)

Write the failing test first, then just enough code to pass it. Red
→ Green → Refactor.

The pyramid

unit (many, fast) → integration (some) → end-to-end (few, slow).
Unit is the foundation.

For UI, render a component and test what a **user** sees with **Testing Library** — that's Sara's integration piece.

▸ // go deeper

Learn more

- vitest.dev/guide — the framework we use.
- [martinfowler.com](https://martinfowler.com/unitTest.html) — “Unit Test” — what counts as a unit.
- [martinfowler.com](https://martinfowler.com/testpyramid.html) — “Test Pyramid” — how the layers fit.
- [xp123.com](https://xp123.com/3A-Arrange-Act-Assert/) — “3A: Arrange, Act, Assert” — Bill Wake's original 2001 post.
- [agileinaflash.blogspot.com](https://agileinaflash.blogspot.com/2005/05/firrst.html) — “F.I.R.S.T.” — Tim Ottinger's original post.
- testing-library.com — testing React the way users use it.

Where the ideas actually come from: Arrange-Act-Assert was named by Bill Wake in 2001; Kent Beck's *Test-Driven Development: By Example* made it popular. FIRST was coined by Tim Ottinger and Brett Schuchert; it's published in Robert C. Martin's *Clean Code*.